Preface: Invalidating Axioms

Axiom

> A statement or proposition which is regarded as being established, accepted, or self-evidently true.

Mathematicians create theories based on axioms, assumptions for things indisputably true. Software architects also build theories atop axioms, but the software world is, well, *softer* than mathematics: fundamental things continue to change at a rapid pace, including the axioms we base our theories upon.

The software development ecosystem exists in a constant state of dynamic equilibrium: while it exists in a balanced state at any given point in time, it exhibits *dynamic* behavior over the long term. A great modern example of the nature of this ecosystem follows the ascension of containerization and the attendant changes: tools like Kubernetes didn't exist a decade ago, yet now entire software conferences exist to service its users. The software ecosystem changes chaotically: one small change causes another small change; when repeated hundreds of times, it generates a new ecosystem.

Architects have an important responsibility to question assumptions and axioms left over from previous eras. Many of the books about software architecture were written in an era that only barely resembles the current world. In fact, the authors believe that we must question fundamental axioms on a regular basis, in light of improved engineering practices, operational ecosystems, software development processes—everything that makes up the messy, dynamic equilibrium where architects and developers work each day.

Careful observers of software architecture over time witnessed an evolution of capabilities. Starting with the engineering practices of Extreme Programming, continuing with Continuous Delivery, the DevOps revolution, microservices, containerization, and now cloud-based resources, all of these innovations led to new capabilities and trade-offs. As capabilities changed, so did architects' perspectives on the industry. For many years, the tongue-in-cheek definition of software architecture was "the stuff

that's hard to change later." Later, the microservices architecture style appeared, where *change* is a first-class design consideration.

Each new era requires new practices, tools, measurements, patterns, and a host of other changes. This book looks at software architecture in modern light, taking into account all the innovations from the last decade, along with some new metrics and measures suited to today's new structures and perspectives.

The subtitle of our book is "An Engineering Approach." Developers have long wished to change software development from a *craft*, where skilled artisans can create one-off works, to an *engineering* discipline, which implies repeatability, rigor, and effective analysis. While software engineering still lags behind other types of engineering disciplines by many orders of magnitude (to be fair, software is a very young discipline compared to most other types of engineering), architects have made huge improvements, which we'll discuss. In particular, modern Agile engineering practices have allowed great strides in the types of systems that architects design.

We also address the critically important issue of *trade-off analysis*. As a software developer, it's easy to become enamored with a particular technology or approach. But architects must always soberly assess the good, bad, and ugly of every choice, and virtually nothing in the real world offers convenient binary choices—everything is a trade-off. Given this pragmatic perspective, we strive to eliminate value judgments about technology and instead focus on analyzing trade-offs to equip our readers with an analytic eye toward technology choices.

This book won't make someone a software architect overnight—it's a nuanced field with many facets. We want to provide existing and burgeoning architects a good modern overview of software architecture and its many aspects, from structure to soft skills. While this book covers well-known patterns, we take a new approach, leaning on lessons learned, tools, engineering practices, and other input. We take many existing axioms in software architecture and rethink them in light of the current ecosystem, and design architectures, taking the modern landscape into account.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic
: Indicates new terms, URLs, email addresses, filenames, and file extensions.

`Constant width`
: Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

`Constant width bold`
Shows commands or other text that should be typed literally by the user.

`Constant width italic`
Shows text that should be replaced with user-supplied values or by values determined by context.

This element signifies a tip or suggestion.

Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download at *http://fundamentalsofsoftwarearchitecture.com*.

If you have a technical question or a problem using the code examples, please send email to *bookquestions@oreilly.com*.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: "*Fundamentals of Software Architecture* by Mark Richards and Neal Ford (O'Reilly). Copyright 2020 Mark Richards, Neal Ford, 978-1-492-04345-4."

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at *permissions@oreilly.com*.

O'Reilly Online Learning

For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, please visit *http://oreilly.com*.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-998-9938 (in the United States or Canada)
707-829-0515 (international or local)
707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at *https://oreil.ly/fundamentals-of-software-architecture*.

Email *bookquestions@oreilly.com* to comment or ask technical questions about this book.

For news and information about our books and courses, visit *http://oreilly.com*.

Find us on Facebook: *http://facebook.com/oreilly*

Follow us on Twitter: *http://twitter.com/oreillymedia*

Watch us on YouTube: *http://www.youtube.com/oreillymedia*

Acknowledgments

Mark and Neal would like to thank all the people who attended our classes, workshops, conference sessions, user group meetings, as well as all the other people who listened to versions of this material and provided invaluable feedback. We would also like to thank the publishing team at O'Reilly, who made this as painless an experience as writing a book can be. We would also like to thank No Stuff Just Fluff director Jay Zimmerman for creating a conference series that allows good technical content to grow and spread, and all the other speakers whose feedback and tear-soaked shoulders we appreciate. We would also like to thank a few random oases of sanity-preserving and idea-sparking groups that have names like Pasty Geeks and the Hacker B&B.

Acknowledgments from Mark Richards

In addition to the preceding acknowledgments, I would like to thank my lovely wife, Rebecca. Taking everything else on at home and sacrificing the opportunity to work on your own book allowed me to do additional consulting gigs and speak at more conferences and training classes, giving me the opportunity to practice and hone the material for this book. You are the best.

Acknowledgments from Neal Ford

Neal would like to thank his extended family, ThoughtWorks as a collective, and Rebecca Parsons and Martin Fowler as individual parts of it. ThoughtWorks is an extraordinary group who manage to produce value for customers while keeping a keen eye toward why things work so that that we can improve them. ThoughtWorks supported this book in many myriad ways and continues to grow ThoughtWorkers who challenge and inspire every day. Neal would also like to thank our neighborhood cocktail club for a regular escape from routine. Lastly, Neal would like to thank his wife, Candy, whose tolerance for things like book writing and conference speaking apparently knows no bounds. For decades she's kept me grounded and sane enough to function, and I hope she will for decades more as the love of my life.